Collabberry HR Agent — QA Verification Document

Environment: Production
Date of this run: 2026-09-07 → ongoing (rolling campaign — each Result carries its own date)
Tester: Aleksa (@aleksaprosperitylabs)
Client witness: _____

1. Systems under test

Health verified 2026-09-07 09:23 UTC via `railway run --service bot bun scripts/health-check.ts` — all six checks PASS.

Component	URL / Handle	Status
Telegram bot	@Coh3erence_hr_bot (id `8811416846`)	PASS — process running, polling Telegram
Frontend	https://collabberry-frontend.vercel.app	PASS — 200
Member signup route	https://collabberry-frontend.vercel.app/member-sign-up	PASS — 200
Backend API	https://backend-production-53b74.up.railway.app	PASS — `POST /api/users/auth/nonce` → 200
Org roster endpoint	`GET /api/orgs/{orgId}`	PASS — returns org "Aleksa"
Data store	Google Sheet `1qM9_Ppm…`	PASS — read OK
Language model (Claude)	`api.anthropic.com`	PASS — 200

Target organisation: "Aleksa", id `63e3ac6c-3e63-4eec-9ac0-de607adf7d05` **Chain:** Arbitrum One (42161) — TeamPoints contract `0x635af529462Fe31cb92C639237207eD7cbAF084e`

> Note: the backend root URL returns 404 by design — there is no root route. Health is confirmed via the API endpoints above, not the bare domain.

Participants for this run

Telegram id	Plays	Notes
`535329585` (Aleksa)	Candidate + org admin	Runs the section B admin commands before applying; the code excludes a contributor from their own reviewer pool, so he cannot review his own proposal
`1971913512` (Simon, @qbfrank)	Reviewer	First half of quorum
`302836662` (Gustavo, @sepu85)	Reviewer (client)	Second half of quorum
`8983159792` (Nikola)	Reviewer (QA tester)	Added 2026-10-14 to make the pool odd. Button-only seat — Approve and Counter, never an admin command. His access is reverted at handover (ledger R6)

The pool grew mid-campaign, and the Result column says which size each case ran at. September runs had two reviewers, `1971913512` and `302836662` — quorum 2 of 2, where majority and unanimity coincide. From 2026-10-14 the pool

is those two plus `8983159792` : **quorum 2 of 3**, which is what the majority-quorum, late-responder and round-cap cases were run at. A contributor is always excluded from reviewing their own proposal, so the candidate never counts toward the pool.

> **Before you start — both reviewers must have opened the bot.** Telegram will not deliver a message to anyone who has never started a chat with the bot, and the failure is silent rather than an error. If either reviewer has not pressed Start at https://t.me/Coh3erence_hr_bot, section E will stall until the 48-hour escalation rather than completing. Confirm both are reachable before beginning.
> If one reviewer cannot participate live, run `bun scripts/remove-admin.ts --id <their id>` to reduce the pool to one reviewer (quorum 1) so the session completes deterministically, and restore the row afterwards. E4 and E5 have since been recorded at a pool of three, so this fallback no longer costs coverage. Note the script rewrites the whole `AuthorizedUsers` block rather than one cell.

2. How to use this document

Each test has a **fixed expected result** agreed in advance. Walk them in order — later tests depend on state created by earlier ones. Record the outcome in the Result column as PASS / FAIL / N-A and put anything surprising in Notes. A FAIL is not necessarily a defect; note it and continue so the session isn't blocked.

> **Backfilled 2026-10-05.** Result columns were filled from the recorded evidence of the September–October QA campaign (sheet rows, timestamps, on-chain records), not from a single sitting. The rows then marked *Not yet run* were executed over the following eleven days and carry their own dates; as of 2026-10-16 none remain.

3. Configured business rules (verified in code, 2026-08-21)

These are the rules the tests below assert against. All are environment-overridable without a code change.

Rule	Configured value	Source
Reviewer response window	48 hours	`REVIEWER_TIMEOUT_HOURS`
Max negotiation rounds	2 — enforced	`MAX_NEGOTIATION_ROUNDS`
Cooldown after rejection	3 days	`COOLDOWN_DAYS`
Reviewer quorum	Majority — `floor(n/2)+1`	`services/quorum.ts`
Hourly → monthly conversion	× 160 h/month (40h × 4wk)	`FTE_HOURS_PER_MONTH`
Fiat portion of comp	0 — remainder in TeamPoints	`DEFAULT_FIAT_REQUESTED`
Contributor statuses	`active` , `hired` , `rejected` , `cooldown`	`models/types.ts`

4. Test cases

A — Access control

ID	Test	Steps	Expected result	Result	Notes
A1	Bot responds	Send `/start` to @Coh3erence_hr_bot	Bot replies; conversation begins	PASS	Both contributors onboarded via `/start` (2026-09-07, 2026-09-23)
A2	Unauthorised user is gated	`/start` from an account not in AuthorizedUsers	Access is refused, no opportunity data leaks	PASS	2026-10-09: 4th account ("Nikola", id 8983159792, no username) sent `/start`; admin DM with name/ID + Contributor/Admin/Ignore buttons received (screenshot). Refusal copy is a static string in the same branch that produced the admin DM (gate.ts) — branch provably executed, nothing dynamic to leak. §26 fix visible: username-less requester rendered without breaking the DM
A3	Admin authorisation	Admin runs `/authorize` for a new Telegram id	User is added and can now proceed	PASS	2026-10-09 authorised via the inline "Authorize as Contributor" button (sheet row `8983159792, contributor` — landed 3×, DEF-5, dedupe queued); 2026-10-11 the user's `/start` cleared the gate and reached `matching` (contributor `c_1791746130193`, session-verified). Note: first post-auth `/start` attempt (2026-10-09) never reached the bot — user had messaged a look-alike bot handle

B — Admin opportunity management

Confirms the admin never edits the spreadsheet directly — all changes go through the bot.

ID	Test	Steps	Expected result	Result	Notes
B1	List opportunities	`/list_opportunities`	Current roles returned with budget and status	**PASS** (2026-10-05)	All three seeded roles returned with commitment, budget and status
B2	Create opportunity	`/add_opportunity`, follow prompts	New role appears in B1 and in the sheet	**PASS** (2026-10-05)	`opp_1791280007760` "Full-Stack Web3 Developer" written to Opportunities sheet, status open. Two defects found live: bare command is a dead end (§25); confirmation reply dropped by Telegram Markdown 400 (§26) — the write succeeded regardless
B3	Edit opportunity	`/edit_opportunity`	Change persists and is reflected in B1	**PASS** (2026-10-06, after fix)	First attempt FAILED 2026-10-06 ~10:10Z (§27: Google 400, 12 values into 11 columns; payload also carried `hourlyRateMin: 0`). K8 fix deployed `12:30:21Z`; re-run ~13:19Z: max 75→80 persisted **with min kept at 50**, and a `status open` edit reopened the paused role — the reopen path §27 also unblocked
B4	Pause opportunity	`/pause_opportunity`	Role stops being offered to new contributors	**PASS** (2026-10-06)	`opp_1791280007760` paused at ~12:38Z, 8 min after the K8 fix deployed — first successful `updateOpportunity` write ever. Confirmation reply arrived; `/list_opportunities` showed 🟡 paused, and a `/start` match run while paused offered only the 3 seeded roles — the paused role was excluded. Reopened afterwards via `status open` (run out of order, before B3/E6)

C — Contributor discovery and matching

ID	Test	Steps	Expected result	Result	Notes
C1	Natural-language intake	Describe skills conversationally, not as a form	Bot extracts skills, rate range, availability, timezone without rigid prompts	PASS	Skills extracted conversationally into both Contributors rows (2026-09-07, 2026-09-23)
C2	Structured capture	Check the Contributors sheet row	Skills, commitment %, rate min/max, timezone, location all populated correctly	PASS	Both rows fully populated — skills, commitment %, rate min/max, timezone, location
C3	Role match	Continue to matching	Bot proposes a relevant role with a match score	PASS	`settlementLikelihood` populated on every agreement row
C4	Poor-fit handling	Offer skills unrelated to any open role	Bot declines gracefully rather than forcing a bad match	FAIL → PASS after fix	2026-10-08 first run FAIL: veterinary-surgery profile offered all three open roles as selectable matches at 11–15% — no match-score floor, decline branch unreachable (§31). Fixed same day (`ea5da9c` , floor at the red-badge boundary of 50), shipped in the Phase E deploy (K10). Re-run post-deploy: "No strong matches found right now. We'll keep your profile on file." — no buttons. Run also exposed §30 (identity regeneration on re-onboarding; repaired, ledger K9)

D — Negotiation

ID	Test	Steps	Expected result	Result	Notes
D1	Counter-offer	Propose a rate above budget	Bot negotiates; settlement likelihood reflects distance from budget	PASS	`opp_002` budget $45–70: ask $75 → likelihood 74; ask $60 → 92
D2	Round cap	Tap "Modify Terms" after a second review	Revision limit reached; only Accept / Walk away are offered — no third round	**PASS** (2026-10-16)	First live run, on `a_1792023040163` (Aleksa + `opp_002`, round 2 of 2, $85/hr). Quorum closed 2 of 3 at 12:23:01Z (Simon `approve`, Nikola `counter 65`); the candidate tapped **Modify Terms** on the resulting counter-offer and the bot refused verbatim: *"We've reached the limit of 2 negotiation rounds for this role. You can accept the reviewers' offer as it stands, or step away — no hard feelings either way."* — leaving only **Accept offer** / **Walk away**, no revision prompt (`resolution.ts:202-215`). The cap reads the persisted `negotiationRound`, not the session, and `nextRoundFromHistory` counts only *reviewed* rows — so the two consumed rounds are `a_1790186286202` ($75/hr, countered to $60) and this one, while the abandoned `a_1790584489474` and superseded `a_1791987837366` drafts correctly consumed nothing
D3	Submission	Accept terms	Agreement row created with status submitted; reviewers notified	PASS	`a_1790186286202` submitted 2026-09-23T17:58:06Z, both reviewers notified

E — Review and quorum

ID	Test	Steps	Expected result	Result	Notes
E1	Reviewer notification	Check reviewer account	Reviewer receives the proposal with approve / counter options	PASS	Gustavo and Simon both received the proposal and acted on it (2026-09-24)
E2	Approve path	Reviewer approves	Decision recorded in ReviewFeedback	PASS	Simon approve recorded 2026-09-24T20:53:20Z
E3	Counter path	Reviewer counters with a different rate	Suggested rate parsed and stored	PASS	Gustavo counter recorded 2026-09-24T09:12:53Z, `suggestedRate = 60` parsed and stored
E4	Majority quorum	With a pool of n reviewers	Review closes at `floor(n/2)+1` responses, not unanimity	**PASS**	2026-10-13, first live run at a pool of 3 (Aleksa/Gustavo/Simon; candidate excluded). `a_1791888028961` closed on the **2nd of 3** responses — Gustavo `counter 75` 13:24:02Z, Aleksa `approve` 17:04:52Z — aggregating to `aggregatedRate 75` with `candidateNotifiedAt 17:05:00Z` while Simon had not responded at all. Unanimity was neither required nor waited for. Defect §32 found in the same run: the counter-offer **prose** misattributes the $75 to the candidate ("your requested rate of $75") — copy only, the stored rate is correct
E5	Late responder	Reviewer responds after quorum	Late response ignored, outcome unchanged	**PASS** (2026-10-15)	First live run, on `a_1791888028961` — the E4 agreement, closed `approved` on 2026-10-13 by Gustavo + Aleksa with Simon never having responded. Simon tapped **Approve** on his original (still-visible) reviewer DM ~2 days after the decision. The bot refused him verbatim: *"This proposal is no longer open for review — the contributor revised it, or it's already been decided. If the revised version needs your input, it'll arrive as a new message."* (forwarded screenshot). ReviewFeedback still holds exactly 2 rows for the agreement — `Gustavo:counter, Aleksa:approve` — so nothing was recorded and the `aggregatedRate 75` outcome is unchanged. Guard: `ensureOpenForReview` (review.ts:99), which also strips the stale keyboard so the dead buttons cannot be tapped twice
E6	48h escalation	Leave a review unanswered past the window	Sweep escalates the agreement (runs every 15 min)	PASS	Fired 2026-10-08T14:15:52Z, inside the predicted 14:06–14:21Z window. Sheet: `a_1791295559074` → `escalated`, no counter-offer generated (aggregation columns empty, correct for no-quorum). Log: `sweepExpiredReviews: a_1791295559074 escalated (no quorum: 0/2)` — pool correctly excludes the candidate; both reviewer DMs dispatched with no send errors logged; Gustavo confirmed receipt 2026-10-09 (forwarded the DM verbatim). Prerequisite drama: the agreement was orphaned hours earlier by §30 and repaired via ledger K9

> **On E4 and E5.** With a two-reviewer pool, `floor(2/2)+1 = 2`, so majority and unanimity coincide and there is no third reviewer left to respond late — neither case is observable in a live run at this pool size. Both are instead evidenced deterministically by the unit tests in `src/services/quorum.test.ts`, which assert the threshold across pool sizes 1–5 (a pool of 3 closing at 2, and of 5 at 3), that a late responder does not change an already-satisfied outcome, and that out-of-pool and repeat votes are not double-counted. Run with `bun test`.

> *Update 2026-10-05:* a fourth, non-admin candidate account is being onboarded, which makes the reviewer pool all three admins (quorum 2 of 3) — both cases become observable live and are scheduled in the closeout plan.
> *Closed 2026-10-15:* both ran live at a pool of 3 and passed — E4 on 2026-10-13, E5 on 2026-10-15 against that same closed agreement. The unit tests remain the evidence for pool sizes we cannot staff.

F — Resolution

ID	Test	Steps	Expected result	Result	Notes
F1	Aggregation	Multiple reviewers give differing rates	Feedback aggregated into a single coherent counter-offer	PASS	`aggregatedRate = 60` + coherent prose in column N, written 2026-09-24T20:53:29Z
F2	Reconciliation	Reviewer counter changes the rate	Final agreement terms match the reviewer's counter, not the original ask	**PASS**	2026-10-14, live, end-to-end with real reviewers — the priority case of the closeout. `a_1791888028961` : candidate asked **$95**, Gustavo countered **75**, candidate tapped Accept, and the contract rate on the row moved **95** → **75** with status `approved` (commitment 50%, 3 months, round 1 preserved). Reconciliation reads `aggregatedRate` , so §32's misworded prose had no effect on the figure agreed. `betaAppAgreementId` empty by design — the candidate was instructed not to open the signup link, so no on-chain agreement was created
F3	Candidate accepts	Accept the resolved offer	Agreement status → approved	PASS	`a_1788962327012` status `approved` (2026-09-09)
F4	Candidate rejects	Decline the offer	Contributor enters 3-day cooldown	**PASS** (2026-10-16)	First live run — no contributor had ever held status `cooldown` before this. Candidate tapped **Walk away** on the capped counter-offer for `a_1792023040163` at 12:33:11Z. Sheet: `c_1790157082079` → status `cooldown` , `cooldownUntil` `2026-10-19T12:33:11.776Z` (exactly `COOLDOWN_DAYS = 3`), and the agreement → `rejected` ; zero rows left `under_review` . Bot replied *"Thank you for your time… You're welcome to re-apply after a 3-day reflection period. We'll keep your profile on file."* (`resolution.ts:243-251` , `closeAsDeclined`). The session was reset, so the candidate cannot re-enter the dead proposal. Reverting the cooldown is ledger **R9**

G — Collabberry integration (the critical path)

ID	Test	Steps	Expected result	Result	Notes
G1	Invite issued	On acceptance	Bot sends a signup link to `/member-sign-up?invitationToken=…`	PASS	`collabberryInviteToken = 06ceee59-545e-452e-a836-5669f9115bf4` (2026-09-09)
G2	Signup completes	Open link, connect a **fresh** wallet, sign up	Account created in Collabberry	PASS	`collabberryUserId = e2a00877-9249-4fcc-a037-2943ba60d534`
G3	Confirmation tap	Return to Telegram, tap "I've signed up"	Bot links the wallet and creates the on-chain agreement	PASS	`walletAddress = 0x290a26a086d4ee4437c80aa1d669576cb7512061`, on-chain agreement created (Arbitrum One)
G4	Sheet updated	Check Contributors row	status `hired`, walletAddress and collabberryUserId populated	PASS	`c_1788807562702` status `hired`, wallet and userId both populated
G5	Agreement recorded	Check Agreements row	`betaAppAgreementId` populated	PASS	`betaAppAgreementId = 231b0916-c607-4fba-9a24-68d593dcfc3c`
G6	Renders in app	Open frontend Team view as org admin	Contributor and agreement visible with correct rate and commitment	PASS	2026-10-07, browser inspection as admin `0xd02f…5878`: Gustavo (Collabberry display name "Guena Cosa") renders as Community Manager, 50% commitment, agreement view/edit icons present. Two prior test identities also visible (July E2E "Aleksa" Frontend Developer $8,000; admin row without agreement) — cleanup is decision D-e
G7	Rate conversion	Compare hourly ask to agreement marketRate	marketRate = hourly × 160	PASS	2026-10-07, same view: Market Rate column reads **$6,400** = $40/hr × 160 (`FTE_HOURS_PER_MONTH`); monetary comp $0 (all TeamPoints, `fiatRequested = 0` default)
G8	Single-use invite	Attempt to reuse a consumed invite token	Second redemption refused cleanly	PASS	2026-10-07, reuse attempted with a fresh wallet (`0x74ec…`) against the consumed G1 token. DB verified: invitation `06ceee59…` unchanged (`isActive=0, usageCount=1/1`) and the attempt was refused org membership (`organization_id=NULL`). Caveat: refusal is server-side only — the frontend renders the full signup form before validating the token, and the attempt leaves an orphaned Users row (`92a9a5cc` , "Coh3rence Tester") behind. Orphan + the unused live invite `87903afd…` (minted 2026-09-14) added to the D-e cleanup list

5. Known limitations — disclose before testing

These are understood behaviours, not defects discovered during the session. Raising them up front avoids them being logged as surprises.

1. **The "I've signed up" tap is manual.** Nothing polls for signup completion. The contributor must return to Telegram and tap the button after finishing in the browser. Tapping *before* signup completes silently does nothing — the contributor stays in `active`. This is the most common point of confusion in a live demo.
2. **Each full end-to-end run needs a fresh wallet.** The backend rejects a second agreement for a wallet that already has one. Re-running G with a previously used wallet will fail by design.
3. **TeamPoints balance shows 0 after hiring.** Minting is a separate manual admin action on-chain and is outside the bot's scope.
4. **The bot has no per-message logging.** State is verified by reading the Google Sheet and backend, not by reading logs.
5. **A bot restart loses the conversation in progress.** Conversation state is held in memory, so a redeploy or crash makes the bot forget what it has already collected and ask for it again. Anything already written to the sheet is safe. Do not redeploy during a session.
6. **Data store is a Google Sheet.** Intentional for the MVP so the client can inspect state directly.

6. Pre-go-live items — not blockers for this session

Configuration that is deliberately relaxed for testing and must be tightened before real contributors are onboarded.

Item	State for this session	Required before go-live
`NODE_ENV`	`production`	Already done
`ALLOW_SELF_REVIEW`	`false` — self-approval is structurally impossible	Already done
Reviewer pool	Three admins (quorum 2 of 3), all reachable — two for the September cases, three from 2026-10-14	Wider pool as the team grows; the QA tester's seat is removed at handover (R6)
Test data	Cleared 2026-09-07 — sheet holds no prior records	Already done

> The self-review escape hatch that previously allowed the whole loop to be driven from one Telegram account has been switched off for this session. Every decision below is made by a distinct participant: the candidate cannot approve their own agreement.

7. Outcome

Tests passed: 32 / 32, campaign closed 2026-10-16. Two needed a fix before they passed, and both histories are kept in the table above rather than overwritten: B3 failed on first run (§27 — every opportunity edit was rejected by Google), fixed and re-run the same day; C4 failed (§31 — no match-score floor made the graceful decline unreachable), fixed and re-run post-deploy. No case is outstanding and no case is failing.

What 32 / 32 does and does not mean. Every row was executed against production, and the Result column carries its own date and evidence. It does **not** mean every path in the code has been seen live: the campaign was built around the 32 agreed cases, and an audit on 2026-10-16 found paths that no case happens to touch. Those are listed below as follow-ups rather than left implicit — none was a known defect, each was an absence of evidence.

The follow-ups were then run, 2026-10-16 to 2026-10-17. Four of the six are closed by live evidence: the cooldown gate (FU-4), the first real **Reject** ever pressed in production and the split-verdict suppression it proves (FU-2), retire-on-commit observed in both halves (FU-3), and the full accept → invite → signup → wallet → on-chain chain re-proven on current code with a fresh wallet (FU-1). The audit's suspicion was right to raise them and wrong about the risk — every path behaved as designed. What the runs *did* surface was a seventh item that no amount of reading would have found: the signup leg only succeeded on a retry (FU-7). **Remaining at handover: FU-5** (a latent silent-failure defect, never observed firing), **FU-6** (an access cleanup), and **FU-7** (an onboarding-friction item whose root cause is still a hypothesis).

Failures / follow-ups:

ID	Issue	Severity	Owner	Action
FU-1	~~The accept → invite → signup → wallet link → on-chain chain was last proven **2026-09-09**. `resolution.ts` changed twice on 2026-10-06 (`6ed1c34` §23, `df642e3` lineage) and both changes sit inside that path~~ **CLOSED 2026-10-17**	**High** — it writes into the client's live Collabberry org	QA	**DONE.** Ran end to end on current code as the continuation of FU-3's round 2: `a_1792257419170` (Community Manager, $38/hr, 25%, 3 months) drew approvals from Nikola and Gustavo, closing quorum 2 of 3 and aggregating to `aggregatedRate 38` / `aggregatedCommitment 25` with *"All reviewers approved."* **Accept** minted the invite, signup completed against a **fresh wallet** **`0xA07a0ee3986b585E0b40a3d35eC5d4Cc90e99fD4`** — deliberately not the org admin `0xd02f…5878` , so the wallet-link step was genuinely exercised — and `betaAppAgreementId f6a5f2d6-1383-4ad7-9157-5bbcfc046fde` was written back with contributor `c_1790157082079` flipped to `hired` . The path the Oct 6 deploys touched is now proven, not assumed. **Two friction points surfaced on the way — see FU-7**
FU-2	~~No reviewer has ever pressed **Reject** in production — every live verdict in the project's history was approve or counter. The split-verdict suppression (§19) rests on unit tests and harness-written feedback rows~~ **CLOSED 2026-10-17**	Medium	QA	**DONE.** `a_1792160941359` (Community Manager, $45/hr, 30%, round 1) drew the first real **Reject** in the project's history — Nikola `8983159792` , *"Not the right fit for this role right now."*, 2026-10-16T15:50:30Z — against Simon's `approve` , closing quorum 2 of 3. Aggregation ran at 2026-10-17T16:13:25.821Z and left `aggregatedRate` **empty**, which is the condition §19 guards: neither reviewer named a figure, so "Accept" could only have meant accepting the candidate's own objected-to rate. The candidate's DM offered **Modify Terms / Walk away only — no Accept button** (confirmed on-device), matching `presentation.ts:97-118` ; the `Accept` keyboard at `presentation.ts:136-141` was never reached. §19 is now evidenced on a live split verdict, not just in unit tests
FU-3	~~§23 retire-on-commit — the behaviour change the client approved (D-d) — shipped unobserved. The one live Modify tap hit the round-cap refusal instead, so the revise-and-resubmit path has no live evidence~~ **CLOSED 2026-10-17**	Medium	QA	**DONE.** Modify tapped on `a_1792160941359` at round 1 of 2 — below the cap, so it reached the revise flow rather than the refusal. Checked in two halves. **Pre-submit:** the revision existed as `a_1792257419170` `draft` ($38/hr, 25%) while the original still read `under_review` with `supersededBy` empty — the standing offer survived the draft, which is the whole point of D-d. **Post-submit** (2026-10-17T17:16:59.377Z): the original flipped to `superseded` with `supersededBy: a_1792257419170` , and the revision went `under_review` at `negotiationRound: 2` . Round 2 — not 3 — confirms `nextRoundFromHistory` counts only reviewed rows. `settlementLikelihood` also recomputed 82 → 86 as the rate moved toward budget. Retire happens on commit, never on draft
FU-4	~~Cooldown is proven as a stored value (F4), not as a gate — no `/start` has been refused by it~~ **CLOSED 2026-10-16**	Low	QA	**DONE.** `/start` sent from `535329585` while `c_1790157082079` held `cooldown` ; the gate refused it verbatim: *"You're currently in a cooldown period. You can re-apply in 3 day(s). This gives both parties time to reflect before the next round."* No "Welcome back", no match list — `gate.ts:49-62`

ID	Issue	Severity	Owner	Action
				returns before the returning-contributor branch, and the day count rounds up correctly against `cooldownUntil 2026-10-19T12:33:11Z`. The gate is now evidenced, not just the stored value
FU-5	`answerCallbackQuery()` runs **after** `handleReview` / `handleResolution` (`bot.ts:143-149`), unlike the `auth:` and `select_opp:` branches which answer first. A throw inside either handler is therefore a silent failure: spinner, no reply, no state change, no trace for the user	Low — latent, not observed firing	Dev	Two-line fix (answer first) plus a deploy in a quiet window, or hand over documented as §34
FU-6	The QA tester (`8983159792`) still holds `admin`, which is also the admin-command gate — write access to the client's live opportunity rows	**High** — access, not behaviour	Dev	Demote that one row to `contributor` before handover (ledger R6)
FU-7	Signup succeeded, but only on a retry. Two things happened first, both observed live on 2026-10-17 and neither reproducible from the sheet afterwards. **(a)** The signup page answered **"no token provided"**. The bot mints `<base>?<param>=<token>` (betaApp.ts:85-87); the code defaults are `https://beta.collabberry.xyz/join` + `invitation` (config.ts:24-29), while the local env resolves `http://localhost:5173/member-sign-up` + `invitationToken` — different path *and* different param name. If Railway has either variable unset it falls through to a default the frontend does not read, which is exactly this symptom. **(b)** MetaMask warned *"This site shows signs of phishing or wallet-draining activity."* Most likely its blocklist reacting to the hosting pattern rather than a real threat, but a real contributor would see the same warning and many would abandon onboarding there	**Medium** — (a) blocks onboarding when it fires, (b) costs candidate trust	Dev	Confirm `BETA_APP_INVITE_URL` and `BETA_APP_INVITE_PARAM` are explicitly set in Railway and match the deployed frontend's route and query key — do not rely on the code defaults. For (b), serve the signup flow from the custom domain rather than a platform-generated subdomain, and if the custom domain is still flagged, file a false-positive removal against MetaMask's `eth-phishing-detect` list. **Unresolved — the invite link's actual domain was never captured during the run, so (a)'s root cause is a hypothesis, not a finding**

Client sign-off: _____ Date: _____